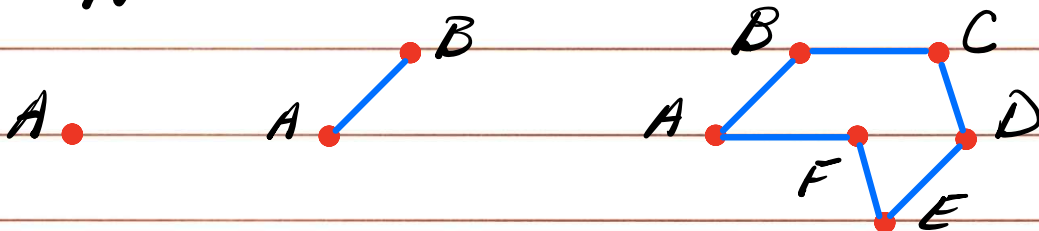


Fundamentals of Computational Geometry

What is Computational Geometry?

Design, analysis and implementation of algorithms for solving geometric problems, e.g. problems involving points, lines segments, polygons, etc.



Applications



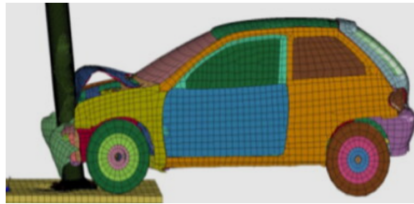
Robotics



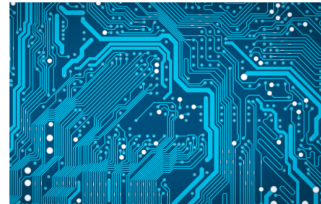
Geographic Information Systems



Computer Graphics & CAD



Scientific Computing

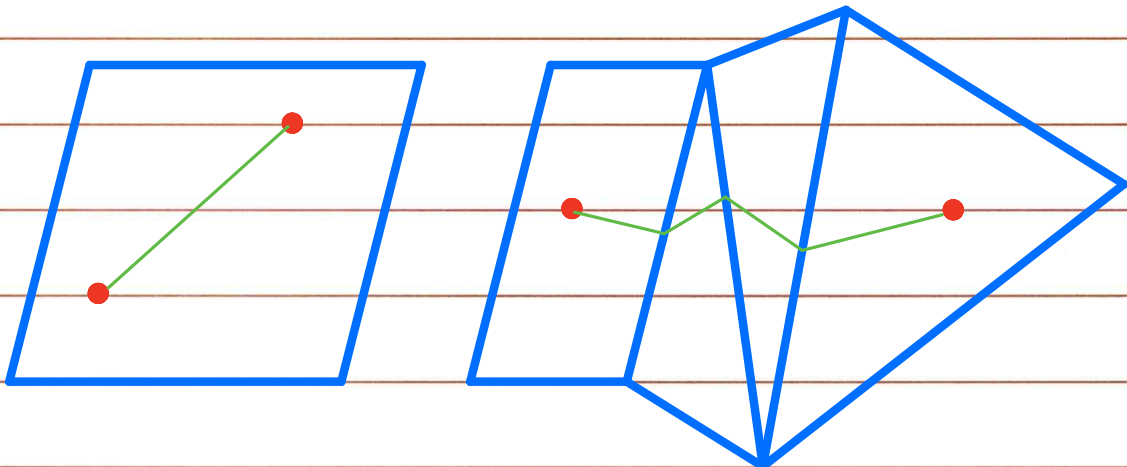


Circuit Design

Common Primitive Objects

2D : Points, Line segments, Polygons, Circles, Ellipses

3D : Points, Line segments, Polygons, Polyhedra,
Spheres, Cylinders, Cones



Fundamental Operations

In computational geometry, the most primitive object is a point.

Common operations on points are

- Addition

- Subtraction

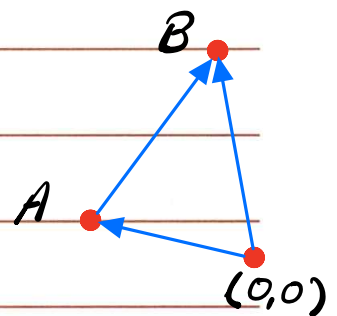
- Dot product

- Cross product

Say $A = (x_1, y_1)$, $B = (x_2, y_2)$

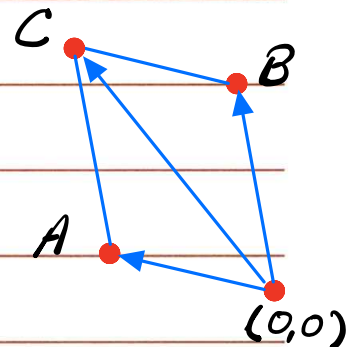
Subtraction:

$$B - A = (x_2 - x_1, y_2 - y_1)$$



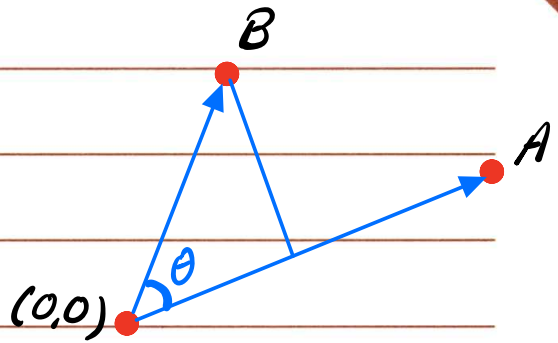
Addition:

$$C = B + A = (x_2 + x_1, y_2 + y_1)$$



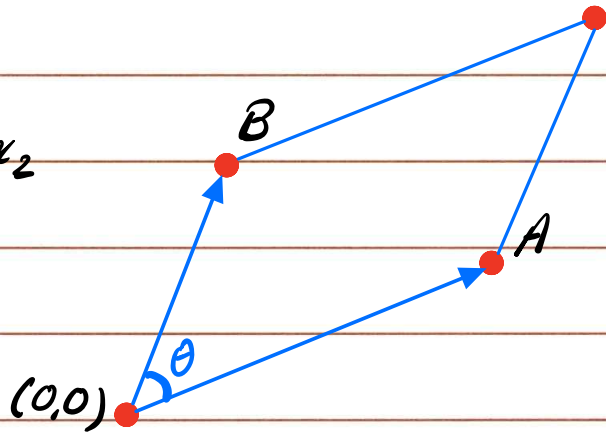
Dot Product:

$$\begin{aligned} A \cdot B &= x_1 x_2 + y_1 y_2 \\ &= |A| \cdot |B| \cos \theta \end{aligned}$$



Cross product:

$$\begin{aligned} A \times B &= x_1 y_2 - y_1 x_2 \\ &= |A| \cdot |B| \sin \theta \end{aligned}$$

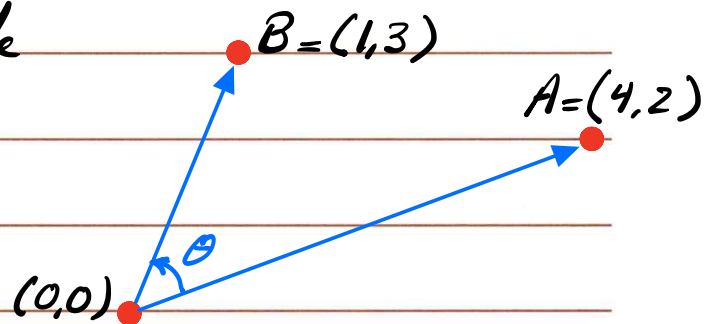


Cross product example

$$\begin{aligned} A \times B &= 4 \times 3 - 2 \times 1 \\ &= 10 \end{aligned}$$

positive value

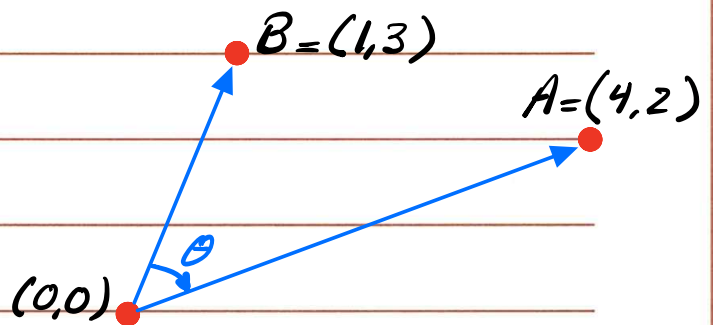
indicates a counterclockwise turn from A to B



$$\begin{aligned} B \times A &= 1 \times 2 - 3 \times 4 \\ &= -10 \end{aligned}$$

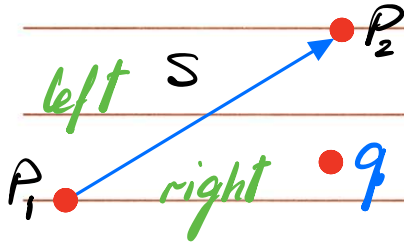
negative value

indicates a clockwise turn from B to A



Line Side Test

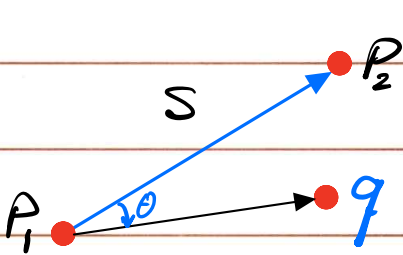
Is point q on the left or right of a line segment $\underline{S}$?



Construct $V_1 = P_2 - P_1$ and $V_2 = q - P_1$

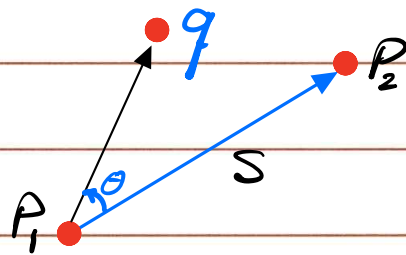
Compute the cross product $V_1 \times V_2$

check if $V_1 \times V_2$ is positive or negative



$$V_1 \times V_2 < 0$$

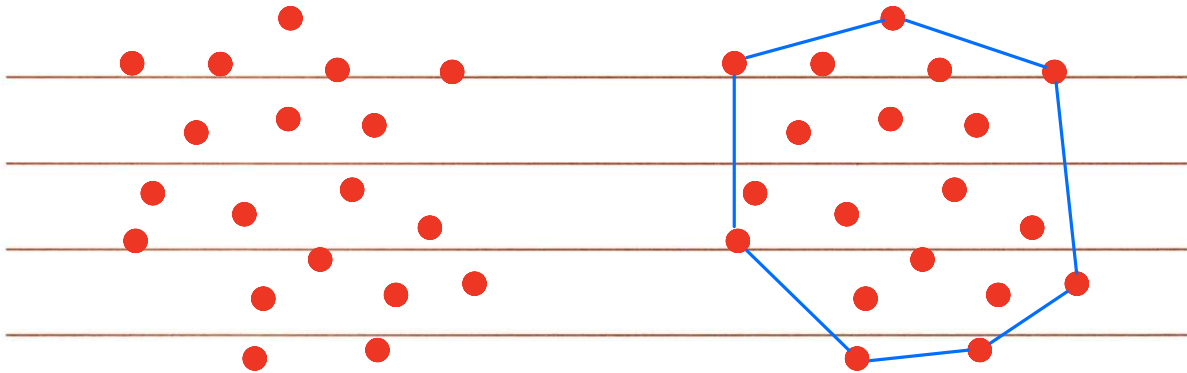
q is to the right of V_1



$$V_1 \times V_2 > 0$$

q is to the left of V_1

Convex Hull



Def.: The convex hull of a set of points P is the smallest convex polygon that contains all of the points in P , either on its boundary or interior.

Applications

Computer graphics

- Collision detection

Robotics

- Path finding

Optimization

- Linear programming

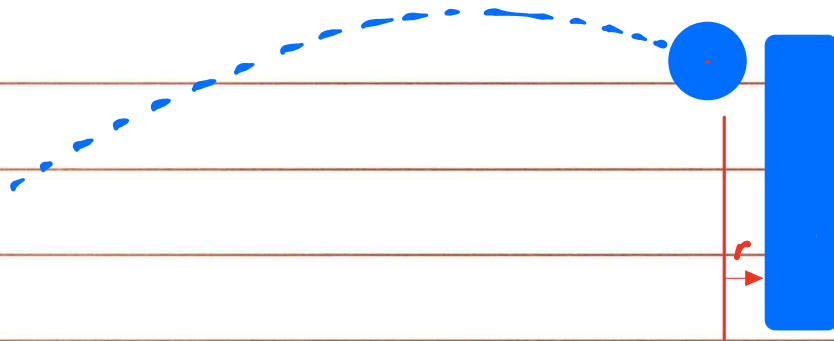
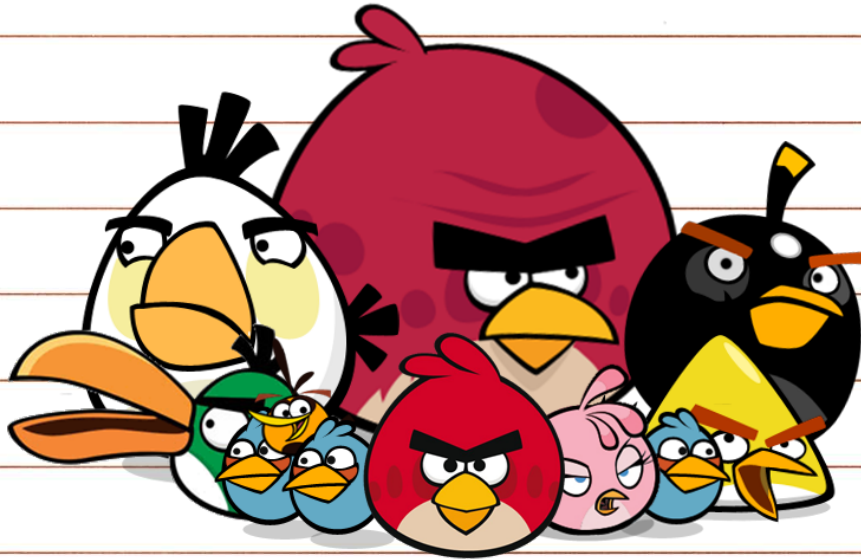
Data science, ML

- Outlier detection

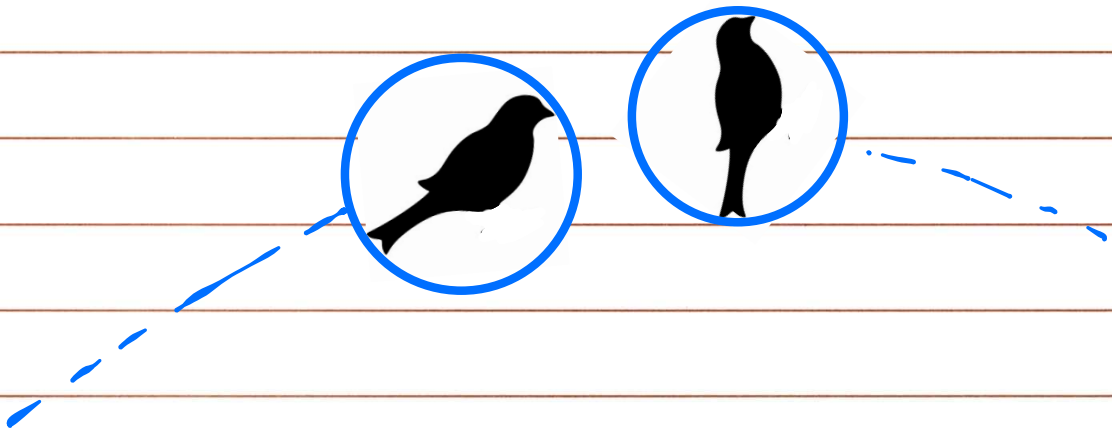
- Clustering

...

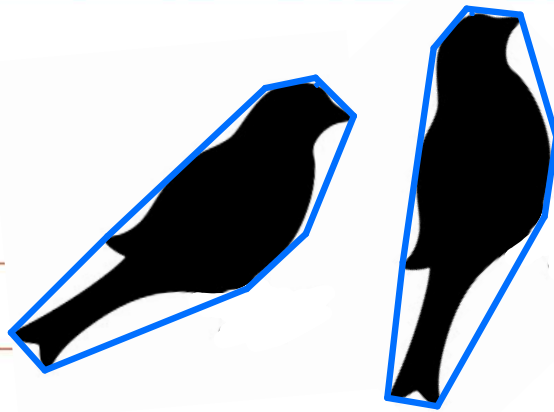
Why are most Angry Birds so round?



It is very easy to detect collisions if objects are circular.



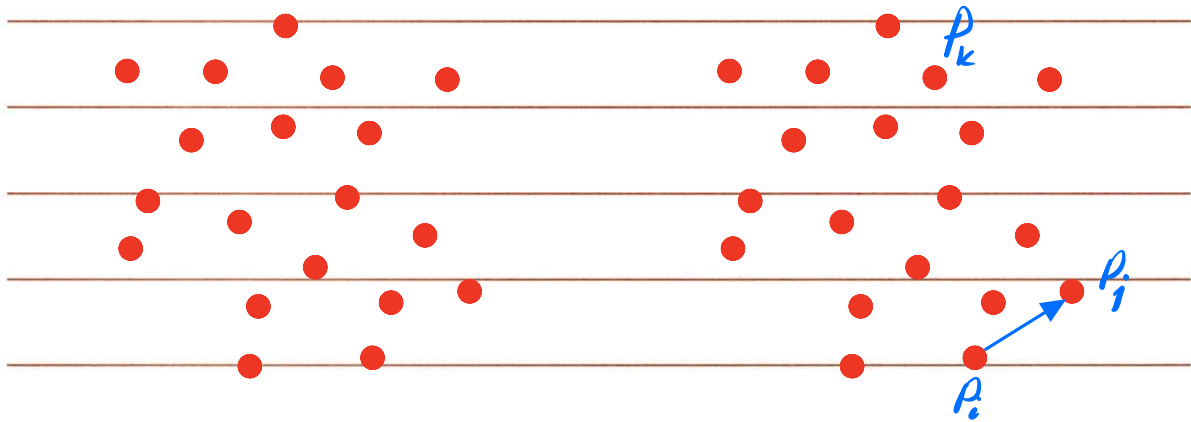
Imagine a game where the colliding objects are non-circular.



Convex hulls can be much better approximations

Intersection of two convex polygons can be found in linear time (WRT the no. of vertices in the two polygons).

Algorithms for 2D Convex Hull



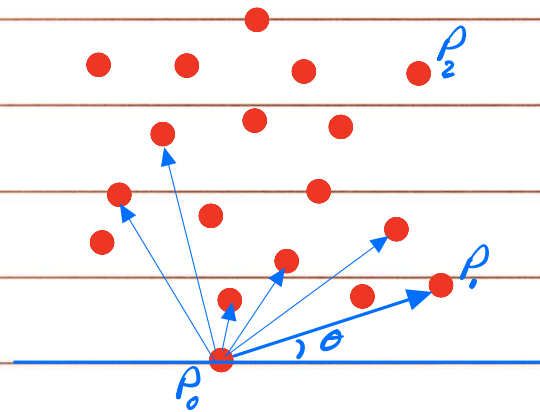
A directed segment between a pair of points $P_i \rightarrow P_j$ is on the convex hull iff all other points are to the left of it.

Brute Force Algorithm

Try every pair of points P_i, P_j
Perform lineSide Test on every other point P_k
If every P_k is on the left then
 Add $P_i \rightarrow P_j$ to the hull
Sort the final set of edges into counter-clockwise order

Complexity? $O(n^3)$

Can we do better?



The lowest y -coordinate point P_0 is always on the convex hull

Take the point with the lowest y -coordinate P_0

Measure the angle from P_0 to all other points

Select the point P_1 with the smallest angle

Add $P_0 \rightarrow P_1$ to the hull

Find point P_2 that has the smallest angle

WRT $P_0 \rightarrow P_1$

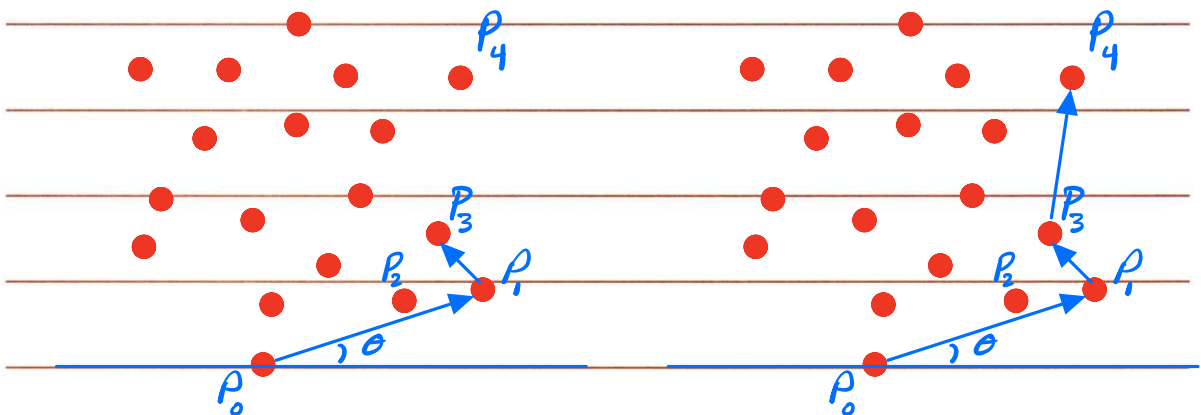
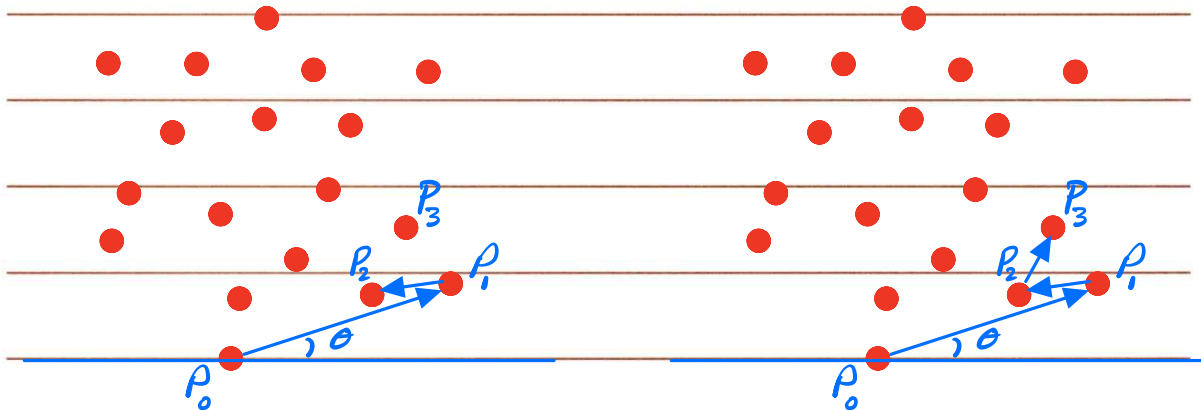
Add $P_1 \rightarrow P_2$ to the hull

Continue until we come back to P_0

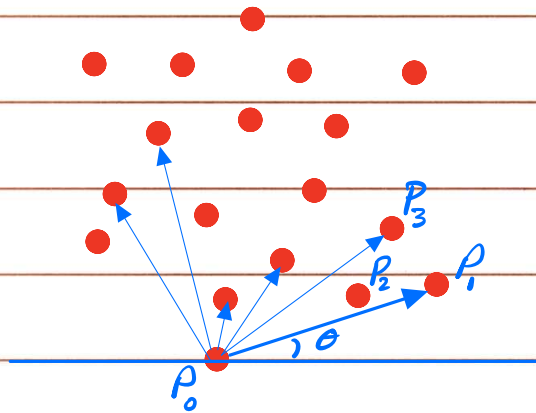
Complexity? $O(n^2)$

Can we do better?

Sort the points counterclockwise by their angle with P_0 and label them $P_1 \dots P_{n-1}$ in that order



Graham Scan



Create empty stack Hull

Push P_0 on Hull

Push P_1 on Hull

Push P_2 on Hull

for $i = 3$ to $n-1$

if P_{i-2} is to the right of $P_{i-1} \rightarrow P_i$ then

Pop Hull

Push P_i on Hull

end for

Complexity? $O(n \lg n)$

Problem Statement

Find the closest pair of points on a Euclidean plane

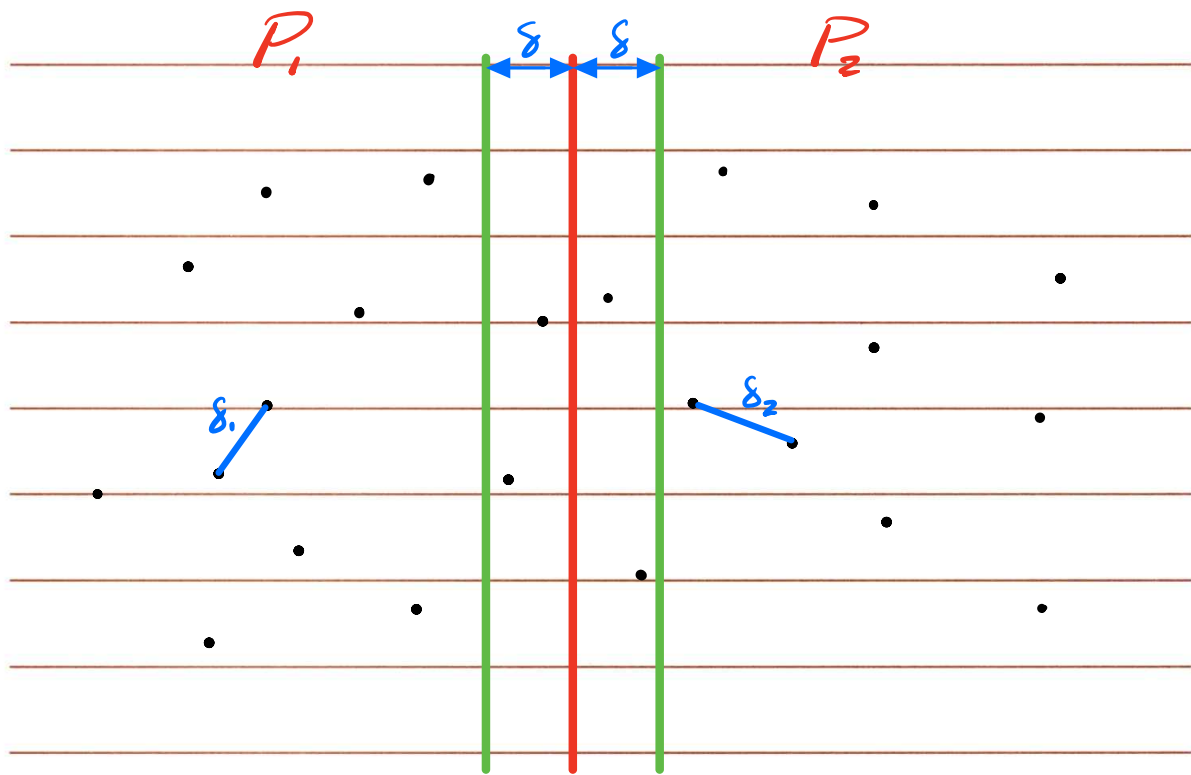


Number of distance calculations and comparisons required:

- Brute force

$$(n-1) + (n-2) + \dots + 1 = O(n^2)$$

- Divide & Conquer ?



Merge Step : Three cases to consider

Case 1 - Both points are in P_1

(found recursively)

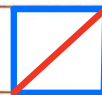
Case 2 - Both points are in P_2

(found recursively)

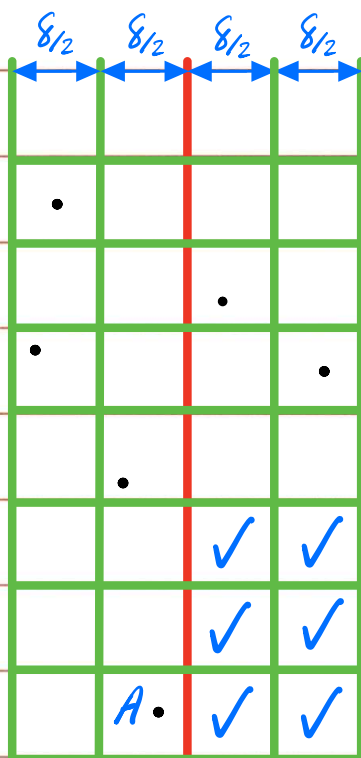
Case 3 - One point is in P_1 and the other in P_2

How do we find these two pts.?

Consider a $8\frac{1}{2} \times 8\frac{1}{2}$ square



Farthest two points within this square are $\frac{8\sqrt{2}}{2}$ away from each other. ($\frac{8\sqrt{2}}{2} < 8$)



✓ candidate cells to look for a pt. that is closer than 8 to A.

Implementation

closest_pair (P)

$O(n \log n)$ Construct P_x : List of pts. sorted by x-coord.

$O(n \log n)$ Construct P_y : List of pts. sorted by y-coord.

$(p_0, p_1) = \text{closest_pair_Rec}(P_x, P_y)$

closest_pair_Rec(P_x, P_y)

if $|P| \leq 3$ then

solve it directly

else

$O(1)$ construct Q_x ... left half of P_x

$O(n)$ construct Q_y ... list of points in Q_x sorted by y-coord.

$O(1)$ construct R_x ... right half of P_x

$O(n)$ construct R_y ... list of points in R_x sorted by y-coord.

$(q_0, q_1) = \text{closest_pair_Rec}(Q_x, Q_y)$

$(r_0, r_1) = \text{closest_pair_Rec}(R_x, R_y)$

$\Theta(1)$ $\delta = \text{Min}(d(q_0, q_1), d(r_0, r_1))$

$\Theta(n)$ { $S = \text{set of points in } P \text{ within distance of } \delta \text{ from } L \text{ (the center line)}$
construct $S_y \dots$ set of points in S sorted by y -coord.

$\Theta(n)$ { for each point $s \in S_y$, compute distance from s to each of the next 11 points in S_y
Let (s, s') be the pair with min. distance
if $d(s, s') < \delta$ then
Return (s, s')
elseif $d(q_0, q_1) < d(r_0, r_1)$ then
 $\Theta(1)$ Return (q_0, q_1)
else
Return (r_0, r_1)
endif

Complexity analysis - Using The Master Theorem

$a=2$, $b=2$, and

The cost of the divide and combine steps = $\Theta(n)$

$$n^{\log_a b} = n^{\log_2 2} = n^1$$

$$f(n) = \Theta(n)$$

⇒ Case 2:

$$T(n) = \Theta(n \lg n)$$

Cost of the preparation step (driver): $\Theta(n \lg n)$

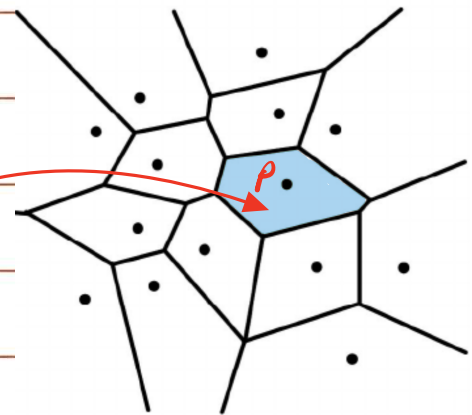
Cost of the divide & conquer solution: $\Theta(n \lg n)$

Overall worst-case time complexity: $\Theta(n \lg n)$

All Nearest Neighbors Problem

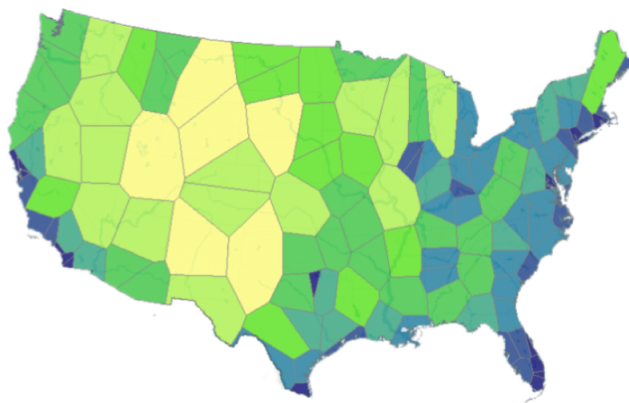
Given n points in the plane, find the nearest neighbors for each point.

Locus of all points on the plane that are closest to point P than any other point

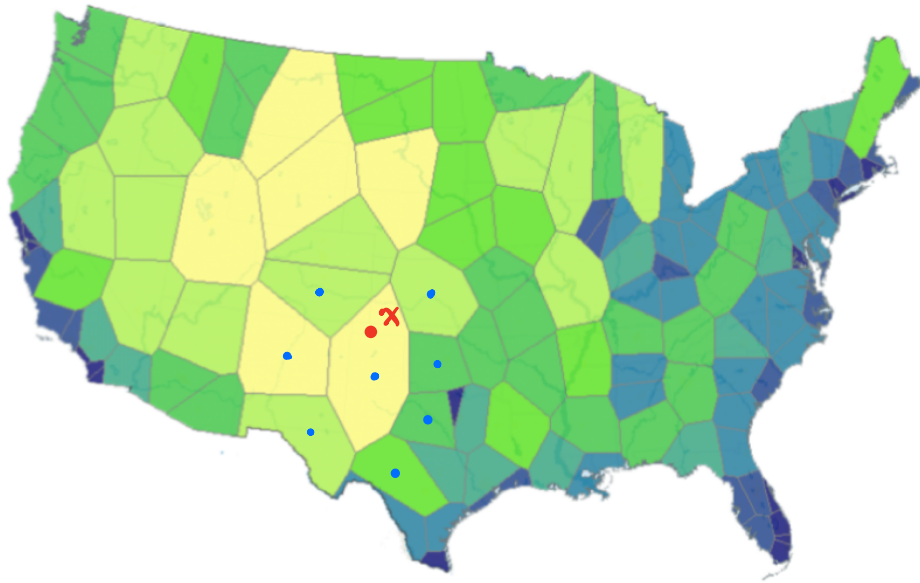


Voronoi Diagrams

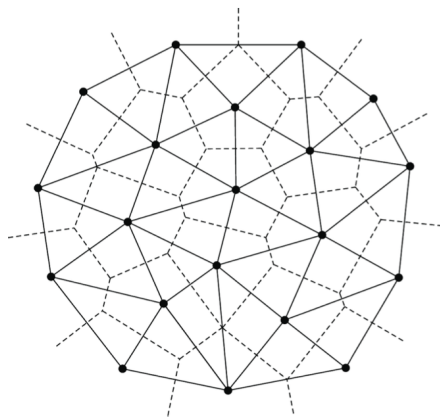
A Voronoi diagram is a partition of a plane into regions close to each of a given set of objects.



Applications



Which airport is closest to point x?

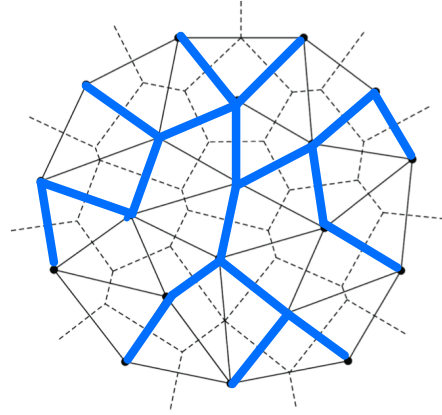


Voronoi Diagram & Delaunay Triangulation

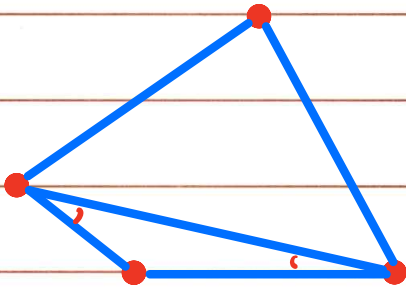
Delaunay Triangulation is a geometric dual of the Voronoi diagram

Applications

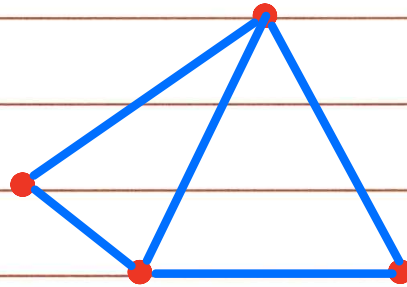
Finding the Euclidean
MST of a set of points.



Producing good tessellations
of objects given their point cloud



One triangle has very
acute angles



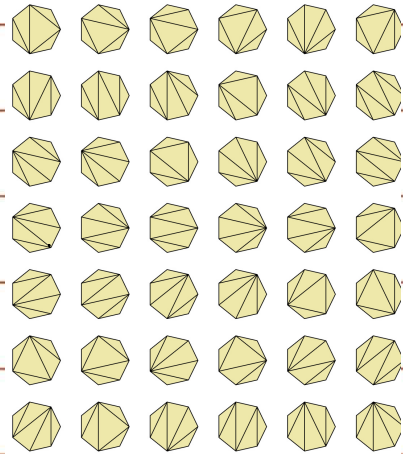
Better triangulation

Delaunay Triangulation maximizes the
minimum angle

Polygon Triangulation

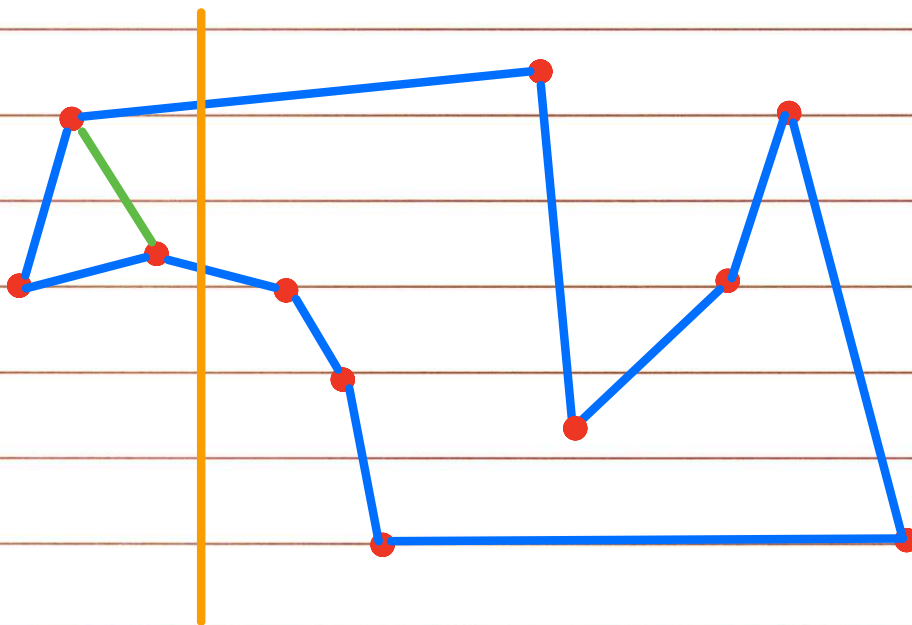
- Trivial to triangulate convex polygons

There are $\frac{n(n-1) \dots (n-2)}{(n-2)!}$ ways to triangulate a convex polygon (n -gon)



From Wikipedia

Monotone Polygon Triangulation



Plane-sweep Method

High level Greedy Algorithm

Create two sorted lists of points
(upper and lower chains) sorted by x-coord.

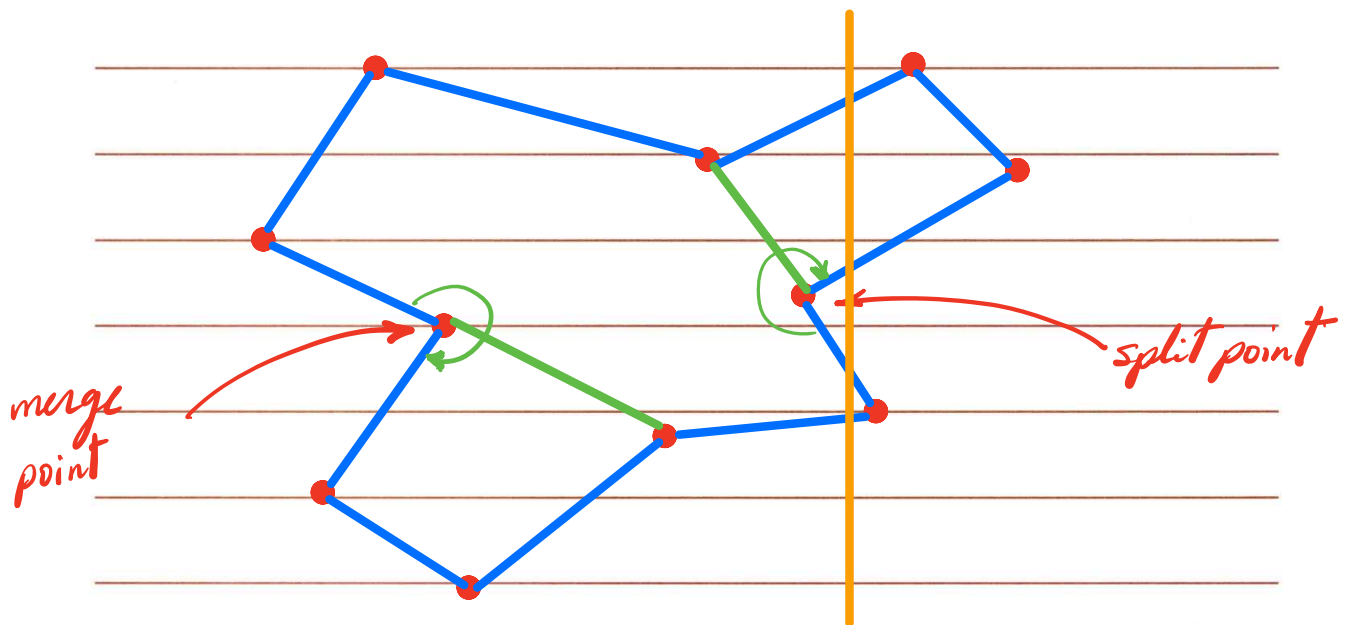
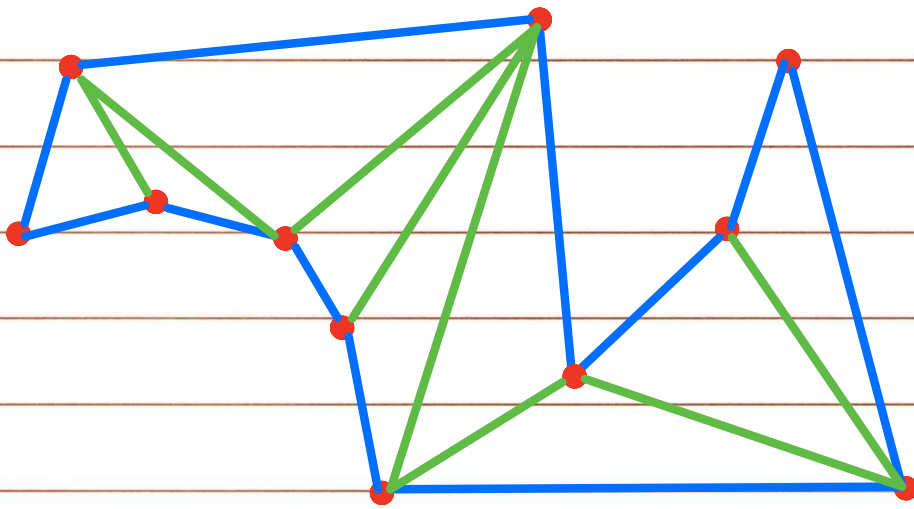
Start from left, choose the next point with
smallest x-coord. from the two lists.

Try to form triangle(s) by connecting
this point with points on the other side
that are to the left of this point.

(This may not always be possible)

Continue until all points are
processed.

Complexity ? $O(n)$



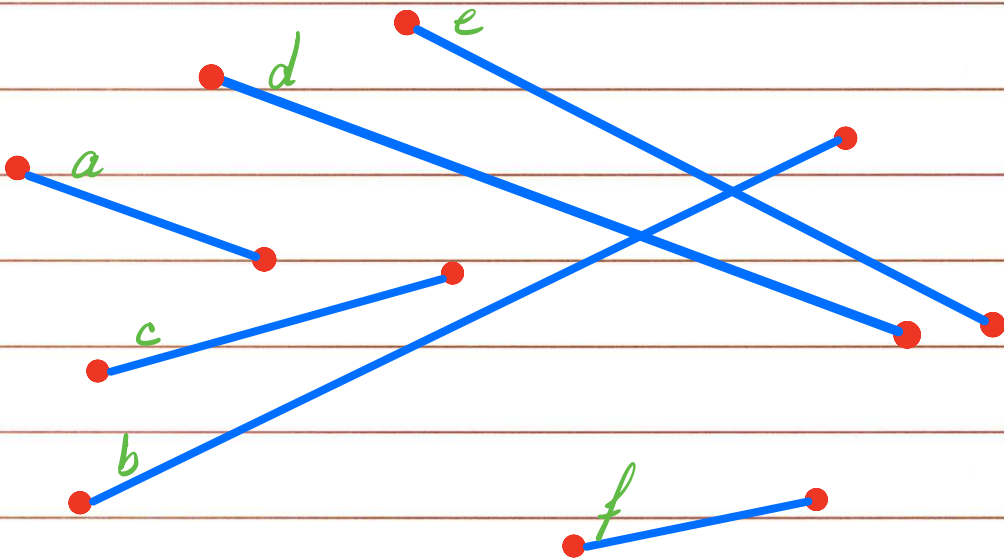
Non-monotone polygons can be subdivided into monotone polygons in $O(n \log n)$ time

Intersection of Line Segments



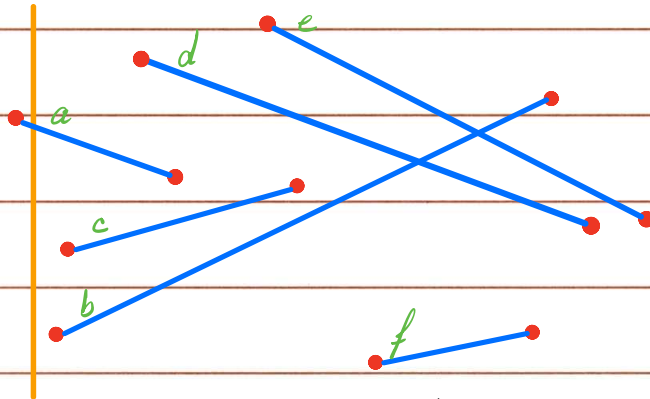
Line segments AB and CD intersect iff A and B are on opposite sides of CD and C and D are on opposite sides of AB.

Intersections of a set of Line Segments



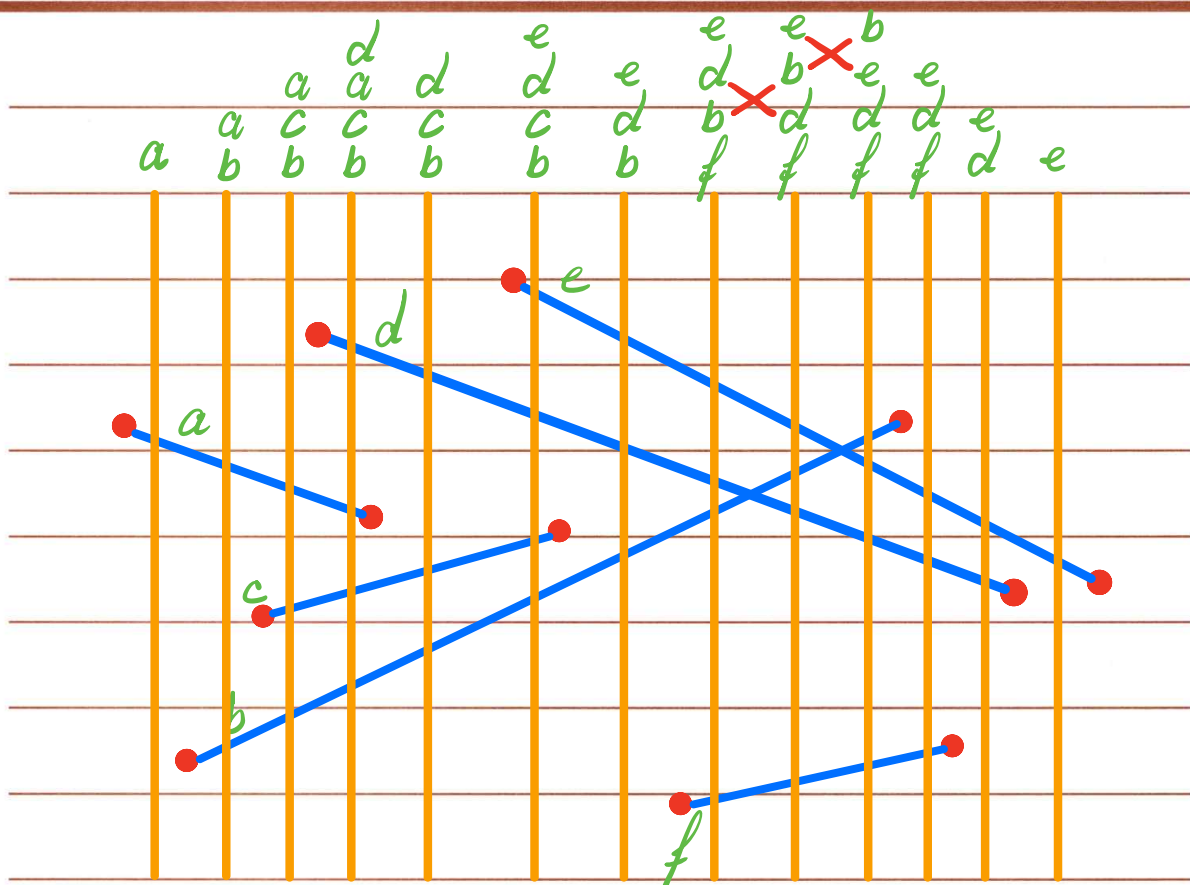
Brute force takes: $O(n^2)$

Plane Sweep Algorithm



Event points are: segment starts, segment ends,
and intersections

Observation: Right before two line segments intersect
They need to be adjacent to each other.



Implementation

Data structures needed

Priority Queue - to store and remove the event points

Balanced binary tree or AVL tree - to store and remove line segments and to keep track of adjacency between line segments.

FindSegmentIntersections(S):

create an empty priority queue EQ

for each segment S , insert its start and end events into EQ

create an empty balanced tree SL

while (EQ is not empty)

$E := \text{deletemin}(\text{EQ})$

if E is start of segment s then

insert s into SL

CheckForIntersect(EQ, s , successor of s)

CheckForIntersect(EQ, s , predecessor of s)

else if E is the end of segment s then

CheckForIntersection(EQ, pred of s , succ of s)

delete s from SL

else if E is cross event for segments s_1 and s_2 then

println " s_1 and s_2 intersect"

remove s_1 and s_2 from SL

re-insert s_1 and s_2 into SL in the opposite order

CheckForIntersection(EQ, s_1 , new neighbor of s_1)

CheckForIntersection(EQ, s_2 , new neighbor of s_2)

Complexity: $O((n+k)\lg n)$

where n = no. of line segments, k = no. of intersections